

YAGNI

You Aren't Gonna Need It

Principle

Build it when you actually need it

DEFINITION

Don't add functionality until it is genuinely required.

YAGNI targets speculative generality — the config switches, plugin points and abstractions added for a future that may never come. It is a statement about timing, not quality: build what today's requirement needs, well, and defer the rest until a real requirement pulls it into existence. It is not licence to skip known near-term needs or basic engineering hygiene.

WHY IT MATTERS

Speculative features are paid for twice — once to build, then forever to carry in tests, docs and cognitive load — and most guessed-at futures either never arrive or arrive shaped differently than imagined. Worse, the speculative abstraction is usually wrong, so when the real need lands you refactor around a bad guess. Deferring keeps the codebase small, fast to change, and honest about what it actually does.

— How it works

Real requirement

A need that exists now — a story, a bug, a committed deliverable. The only legitimate trigger for writing code.

Speculative need

A "we might want..." future with no current driver. YAGNI says defer it — record it, don't build it.

Refactor moment

When a speculative need becomes real (the second variant arrives). That is when you introduce the abstraction — now informed by facts.

HOW TO APPLY

1. For each abstraction or option, ask: what current requirement forces this? No answer means don't build it.
2. Implement the concrete thing today's story needs, cleanly.
3. Write deferred ideas down in a backlog instead of coding them.
4. When a real second case appears, refactor toward the abstraction (the rule of three).

LITMUS TEST

Is there a real, current requirement for this code path? If the only justification is "we might need it later", defer it.

— Smell → fix

A pluggable channel system built for a single, hypothetical future:

```
class Notifier {  
  channels = [] // 'pluggable' for an imagined future  
  send(msg) { for (const c of this.channels) c.deliver(msg) }  
}  
// only email exists; the abstraction earns nothing yet
```

↓ DEFER UNTIL THE NEED IS REAL

```
class Notifier {  
  send(msg) { sendEmail(msg) } // the one channel we actually have  
}  
// add the seam when the SECOND channel really arrives
```

Start concrete. When the second channel appears, refactor toward the abstraction (the rule of three) — now driven by a real requirement, not a guess.

USE WHEN

- ✓ Tempted to add "future-proof" config or flags
- ✓ Premature generalization

AVOID WHEN

- ▲ Skipping genuinely-known near-term needs or basic extensibility

— Trade-offs & pitfalls

PROS

- + Less code and maintenance
- + Faster delivery

CONS

- Some change is costly to retrofit
- Needs judgment

COMMON PITFALLS

- ▲ Using YAGNI to skip genuinely-known near-term needs or basic hygiene (logging, error handling) — that is negligence, not YAGNI.
- ▲ Building a framework for one use case "to save time later".
- ▲ Adding config flags and extension points nobody asked for, then maintaining them forever.

WHERE YOU SEE IT

- Shipping one notification channel (email) instead of a pluggable channel system on day one.
- A concrete function now, promoted to a Strategy only when a second algorithm appears.
- Skipping a generic multi-tenant layer until a second tenant is actually contracted.

SEE ALSO

KISS	deferring speculative work is the main way to keep code simple
OCP	introduce the extension seam when the second variant actually exists
DRY	don't abstract for reuse that doesn't exist yet
Col	prefer the smallest composition that meets today's need

— Interview & sources

YAGNI vs OCP — contradiction?

They balance. YAGNI says do not pre-abstract; OCP says be open at the axis that varies. Resolve with the rule of three: refactor to OCP when the variation actually appears.

Where does YAGNI not apply?

To genuinely known near-term needs and basic hygiene — logging, error handling, sane boundaries. YAGNI targets speculation, not professionalism.

What cost is YAGNI fighting?

Carrying cost: every speculative feature is code to maintain, test, document and reason about — paid every day until it is deleted.

How do YAGNI, KISS and DRY relate?

They pull together: YAGNI removes speculative scope, KISS keeps what remains simple, DRY keeps its knowledge in one place. YAGNI is about when to build; the others about how.

REMEMBER

Build it when the requirement is real, not when you imagine you might need it.

SOURCES

Kent Beck — "Extreme Programming Explained" (1999): YAGNI as an XP practice

Martin Fowler — "Yagni" (2015): the cost of building presumptive features